



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Компилятор языка Pascal»

Студент ИУ7-22М
(Группа)

(Подпись, дата)

А. Е. Лахов
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

А. А. Ступников
(И. О. Фамилия)

2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Структура компилятора	4
1.1.1 Препроцессор	5
1.1.2 Лексические и синтаксические анализаторы	5
1.1.3 Семантический анализатор	7
1.2 Генераторы лексических и синтаксических анализаторов	7
1.3 Генерация исполняемого кода	8
1.4 Особенности языка Pascal	10
1.4.1 Типизация	11
2 Конструкторская часть	13
2.1 Постановка задачи	13
2.1.1 Поддерживаемые типы данных	14
2.2 Построение анализаторов исходного кода	15
2.3 Построение промежуточного представления (IR)	16
3 Технологическая часть	19
3.1 Выбор средств программной реализации	19
3.2 Сгенерированные классы анализаторов	19
3.3 Тестирование компилятора	20
3.4 Пример программы инверсии списка	20
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25
ПРИЛОЖЕНИЕ А. Грамматика языка Pascal	26

ВВЕДЕНИЕ

Компилятор – программное обеспечение, которое переводит исходный текст программы на некотором языке программирования в машинный код (или же некоторый низкоуровневый код) для исполнения на некотором устройстве, под которое и пишется компилятор. В процессе преобразования исходного кода выполняется его оптимизация и анализ ошибок, что позволяет улучшить производительность и избежать некоторых сбоев при выполнении машинных команд [1].

Язык Pascal был создан с целью обучения программированию и алгоритмическому мышлению. До сих пор многие учебные заведения используют его как один из первых языков, чтобы познакомить студентов с основами программирования. Разработка современного компилятора для Pascal может предоставить студентам улучшенный инструментарий и увеличенные возможности для практики и исследования. [2].

Цель курсовой работы – разработка компилятора языка Pascal. Компилятор должен выполнять чтение текстового файла, содержащего код на языке Pascal и генерировать на выходе программу, пригодную для запуска.

Для достижения цели необходимо решить следующие задачи:

- проанализировать грамматику языка Pascal, выделить ее основные составляющие;
- изучить существующие средства для анализа исходных кодов программ, системы для генерации низкоуровневого кода, запуск которого возможен на большинстве из используемых платформ и операционных систем;
- реализовать прототип компилятора языка Pascal.

1 Аналитическая часть

1.1 Структура компилятора

Структурно компилятор состоит из трёх основных частей *Frontend*, *Middle-end* и *Backend*.

Frontend компилятора отвечает за анализ исходного кода программы. Данный компонент обычно включает в себя препроцессор, лексические, синтаксические и семантические анализаторы, генераторы промежуточного представления.

Middle-end выполняет оптимизации и преобразования программы, основанные на промежуточном представлении, полученном от *Frontend*. Данный компонент может включать в себя различные оптимизации, такие как удаление недостижимого кода, упрощение выражений, встраивание функций. *Middle-end* также может выполнять анализ зависимостей, определение времени жизни переменных и иные анализы, необходимые для оптимизации программы. После завершения работы *Middle-end* создает промежуточное представление программы, которое будет передано *Backend* для генерации машинного кода.

Backend компилятора отвечает за генерацию машинного кода на основе промежуточного представления программы, полученного от *Middle-end*. Данный компонент анализирует промежуточное представление и генерирует соответствующий машинный код для целевой архитектуры. *Backend* также может выполнять дополнительные оптимизации, специфичные для целевой архитектуры, такие как распределение регистров и сокращение размера кода. После завершения работы *Backend* генерирует исполняемый файл, который может быть запущен на целевой аппаратной платформе [3].

На рисунке 1.1 изображена упрощенная схема работы компилятора в нотации IDEF-0.

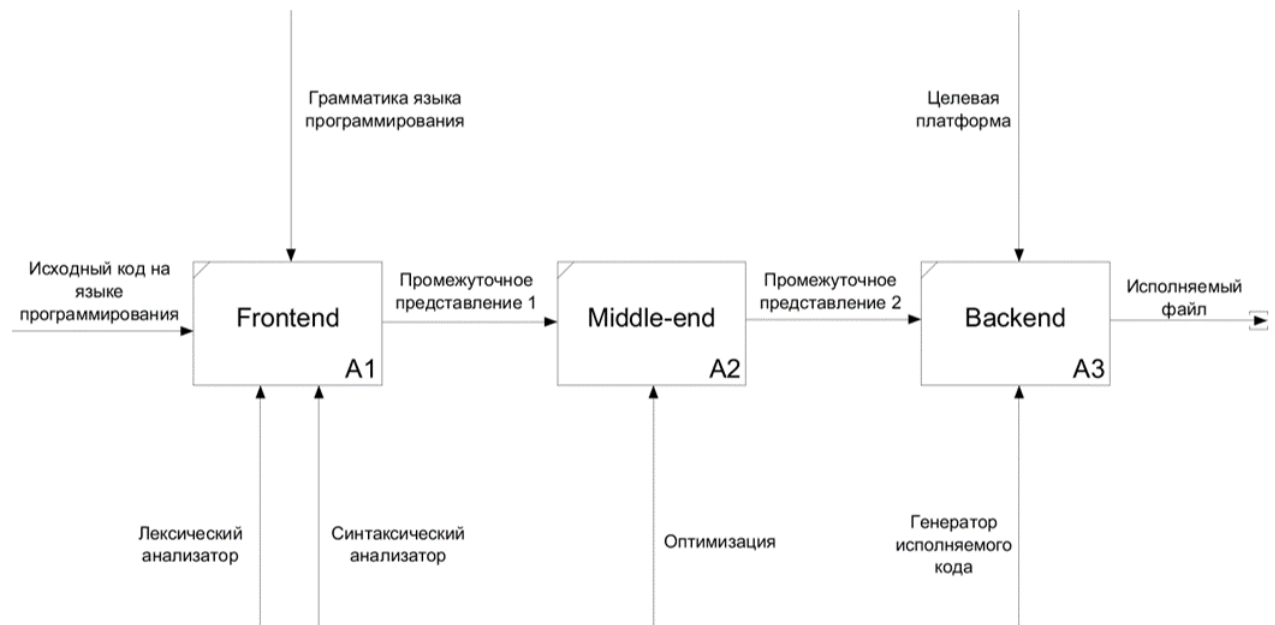


Рисунок 1.1: Упрощенная схема работы компилятора

1.1.1 Препроцессор

Препроцессоры создают входной поток информации для компилятора, выполняют функции:

- обработка макросов;
- включение файлов;
- обработка языковых расширений.

1.1.2 Лексические и синтаксические анализаторы

Лексический и синтаксический анализаторы являются ключевыми компонентами компилятора, ответственными за преобразование исходного кода программы в промежуточное представление, которое необходимо для дальнейшей обработки.

Лексический анализатор (*лексер*) – преобразует поток символов исходного кода программы в последовательность токенов (рис 1.2). Токены представляют собой лексемы, или элементы языка, такие как идентификаторы,

ключевые слова, операторы, числа и строки. Лексический анализатор также может игнорировать комментарии и обрабатывать различные форматирования, такие как пробелы и переводы строк [4].

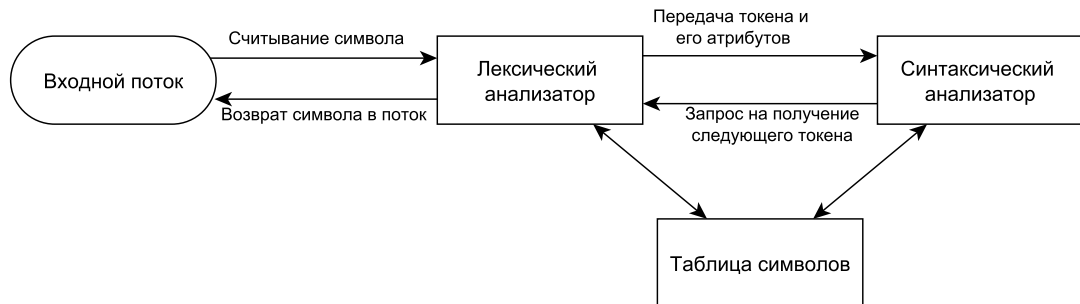


Рисунок 1.2: Лексический анализатор.

Синтаксический анализатор (*парсер*) – анализирует структуру программы на основе последовательности токенов, созданных лексическим анализатором (рис 1.3). Он использует грамматику языка программирования для определения правил, по которым могут комбинироваться токены, чтобы создавать корректные конструкции языка. Синтаксический анализатор создает синтаксическое дерево (дерево разбора), которое представляет собой иерархическое представление структуры программы [5].

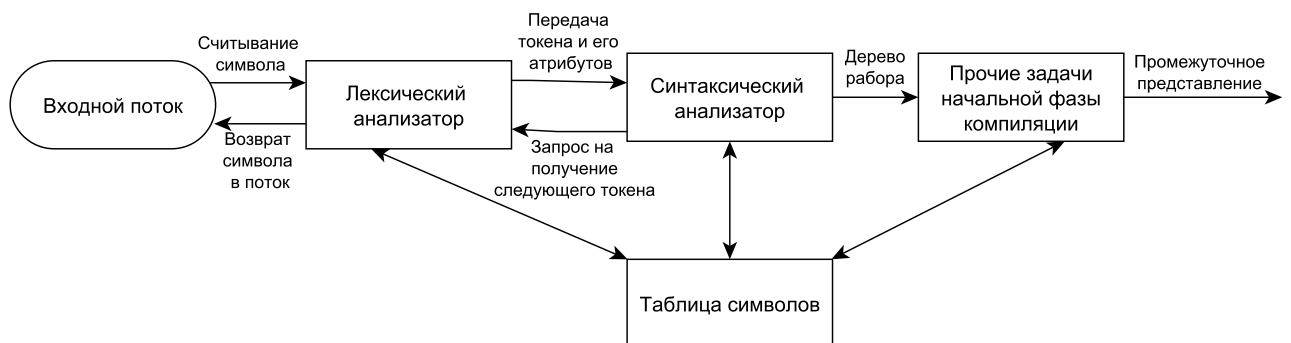


Рисунок 1.3: Синтаксический анализатор.

Существует несколько подходов для реализации лексического и синтаксического анализаторов:

- с использованием стандартных алгоритмов анализа;
- с привлечением готовых инструментов генерации.

1.1.3 Семантический анализатор

В процессе семантического анализа проверяется наличие семантических ошибок в исходной программе и накапливается информация о типах для следующей стадии – генерации кода. Используются иерархические структуры, полученные во время синтаксического анализа для идентификации операторов и операндов выражений и инструкций.

Как правило, семантический анализатор разделяется на ряд более мелких, каждый из которых предназначен для конкретной конструкции. Соответствующий семантический анализатор вызывается синтаксическим анализатором как только он распознает синтаксическую единицу, требующую обработки.

Семантические анализаторы взаимодействуют между собой посредством информации, хранящейся в структурах данных, например, в таблице символов.

1.2 Генераторы лексических и синтаксических анализаторов

Lex [6] – стандартный инструмент для получения лексических анализаторов в операционных системах Unix, обычно используется совместно с генератором синтаксических анализаторов Yacc. В результате обработки входного потока получается исходный файл на языке Си. Flex (Fast Lexical Analyzer) [7] заменяет Lex в системах на базе пакетов GNU и имеет аналогичную функциональность. Bison – аналогичный YACC генератор для GNU систем.

ANTLR (ANother Tool for Language Recognition) [8] – генератор лексических и синтаксических анализаторов, позволяет создавать анализаторы на многих языках программирования, строить и обходить деревья синтаксического анализа с использованием паттернов посетитель или слушатель.

Coco/R – инструмент генерации компиляторов или интерпретаторов языка. Coco/R изначально разрабатывался в ETHZ и перешел вместе с Ханспетером Мессенбеком в Университет Линца. Coco/R распространяется на условиях слегка смягченной GNU General Public License [9].

Сравнительный анализ генераторов

В таблице 1.1 представлен сравнительный анализ анализаторов исходного кода.

Таблица 1.1: Сравнение генераторов анализаторов исходного кода

Критерии	ANTLR	Flex + Bison	Coco/R
Алгоритм лексера	ДКА	ДКА	ДКА
Алгоритм парсера	LL(*)	LR	LL(1)
Поддерживаемые языки программирования	C#, Java, Python, JavaScript, C++, Swift, Go, PHP	C, C++, Java	C, C++, C#, F#, Java, Ada, Object Pascal, Delphi, Modula-2, Oberon, Ruby, Swift, Unicon, Visual Basic
Входная грамматика	РБНФ	Yacc	РБНФ
Интеграция с инструментами разработки	IntelliJ IDEA, Eclipse и другие	Нет	Нет
Наличие встроенных инструментов отладки	Да	Нет	Нет
Лицензия использования	BSD-подобная	GNU GPL	GNU GPL

1.3 Генерация исполняемого кода

Последний этап – генерация кода. Начинается тогда, когда во все системные таблицы занесена необходимая информация. Компилятор переходит к построению соответствующей программы в машинном коде. Код генерируется при обходе дерева разбора, построенного на предыдущих этапах.

Для получения машинного кода требуется два отдельных прохода:

- генерация промежуточного кода;
- генерация собственно машинного кода.

Для каждого узла дерева генерируется соответствующий операции узла код на целевой платформы. В процессе анализа кода программы данные связываются с именами переменных. При выполнении генерации кода предполагается, что вход генератора не содержит ошибок. Результат – код, пригодный для исполнения на целевой платформе.

Платформы для разработки компиляторов

Виртуальная машина – это абстрактная вычислительная машина, которая исполняет программный код, представленный в виде некоторого промежуточного представления – байткода. Виртуальные машины обычно используются для исполнения программного кода, который был предварительно скомпилирован в промежуточное представление, а не в машинный код конкретной архитектуры процессора. Примерами виртуальных машин являются Java Virtual Machine (JVM), Common Language Runtime (CLR), и V8 (WASM) для веб-приложений .

LLVM (*Low Level Virtual Machine*) [10] – представляет собой инфраструктуру для разработки компиляторов и связанных инструментов. Он включает в себя мощный оптимизатор и генератор машинного кода, который может работать с различными архитектурами процессоров. LLVM генерирует промежуточное представление LLVM IR, которое затем транслируется в машинный код для целевой архитектуры. Он также поддерживает генерацию байткода для виртуальных машин, таких как JVM и WASM.

GCC (*GNU Compiler Collection*) – это другая широко используемая инфраструктура для разработки компиляторов. В качестве промежуточного представления используется GIMPLE. GCC генерирует машинный код для различных архитектур процессоров и поддерживает генерацию байткода для некоторых виртуальных машин.

Java ASM (*Java bytecode manipulation framework*) – это библиотека на языке Java, предназначенная для манипулирования байткодом Java. Ее можно использовать для модификации существующих классов или для динамиче-

ской генерации классов непосредственно в форме байткода. ASM предоставляет некоторые распространенные преобразования байт-кода и алгоритмы анализа, на основе которых могут быть созданы пользовательские сложные преобразования и инструменты анализа кода. ASM предлагает функциональность, аналогичную другим фреймворкам байт-кода Java, но ориентирован на производительность.

ILAsm (*Intermediate Language Assembler*) – это инструмент для создания исполняемых программ для платформы CLR (*Common Language Runtime*), использующей байткод Intermediate Language (IL). Он позволяет разработчикам создавать IL код вручную и компилировать его в исполняемый байткод с помощью утилиты компиляции IL@.

WASM (*WebAssembly*) – это низкоуровневый бинарный формат, предназначенный для исполнения в веб-браузерах. Binaryen – это инструмент для манипулирования и оптимизации байткода WebAssembly. Он обеспечивает высокую производительность и безопасность и может быть использован для исполнения приложений на различных языках программирования в веб-среде.

В данной работе использование инструмента LLVM для генерации кода на целевую платформу x64 было обусловлено его мощными возможностями и гибкостью. LLVM предоставляет инфраструктуру для разработки компиляторов и связанных инструментов, включая оптимизатор и генератор машинного кода, способный работать с различными архитектурами процессоров.

1.4 Особенности языка Pascal

Программа на Pascal состоит из модулей, которые могут быть использованы для организации кода. Каждый файл программы должен начинаться с объявления программы (которая выполняется самостоятельно) или вспомогательного модуля (который может быть использован в другом модуле или программе, но не может исполняться самостоятельно). Подключение вспомогательных модулей выполняется с помощью ключевого слова `uses`.

Переменные объявляются с использованием ключевого слова `var`, за которым следует имя переменной (или их список) и через двоеточие ее (их) тип.

Функции объявляются с использованием ключевого слова `function`, за которым следует имя функции, в круглых скобках указывается список параметров и через двоеточие – тип возвращаемого значения. Процедуры (в отличие от функций не возвращают никакого значения) объявляются с использованием ключевого слова `procedure`, за которым следует имя функции, в круглых скобках указывается список параметров.

Pascal поддерживает стандартные управляющие конструкции, такие как условные операторы `if`, циклы `for`, `switch` и операторы `break`, `continue` для управления выполнением программы.

1.4.1 Типизация

В статически типизированных языках типы данных определяются на этапе компиляции. Переменные должны быть объявлены с указанием их типа, и этот тип не может быть изменен во время выполнения программы. Проверка типов происходит на этапе компиляции, что позволяет выявлять множество ошибок ещё до запуска программы.

В динамически типизированных языках типы данных определяются во время выполнения программы. Переменные могут содержать значения разных типов, и их тип может изменяться во время выполнения программы. Проверка типов происходит во время выполнения программы, что может приводить к ошибкам во время выполнения, если типы несовместимы.

В языке программирования Pascal используется статическая типизация. В Pascal типы данных указываются явно при объявлении переменных, функций и других элементов программы.

Pascal предоставляет следующие встроенные типы данных:

- числовые типы (`integer`, `real`);
- строковые типы (`string`);
- булев тип (`boolean`);

- указатели (T);
- массивы (`array[n] of T`);
- структуры (`record`).

В Pascal можно создавать пользовательские типы данных с помощью ключевого слова `type`. Это позволяет абстрагировать сложные структуры данных и создавать собственные абстракции. В Pascal у каждого типа данных есть свое нулевое значение. Например, для числовых типов это 0, для строк это пустая строка, а для указателей это `nil`.

2 Конструкторская часть

2.1 Постановка задачи

Необходимо разработать прототип компилятора языка программирования Pascal. Для генерации парсеров и лексеров грамматики языка необходимо использовать генератор синтаксических анализаторов ANTLR4. Для построения IR необходимо использовать библиотеку LLVM. Для компиляции IR для целевой архитектуры x64 также необходимо использовать LLVM. На рисунках 2.1-2.2 представлены IDEF0 диаграммы компилятора языка программирования Pascal.

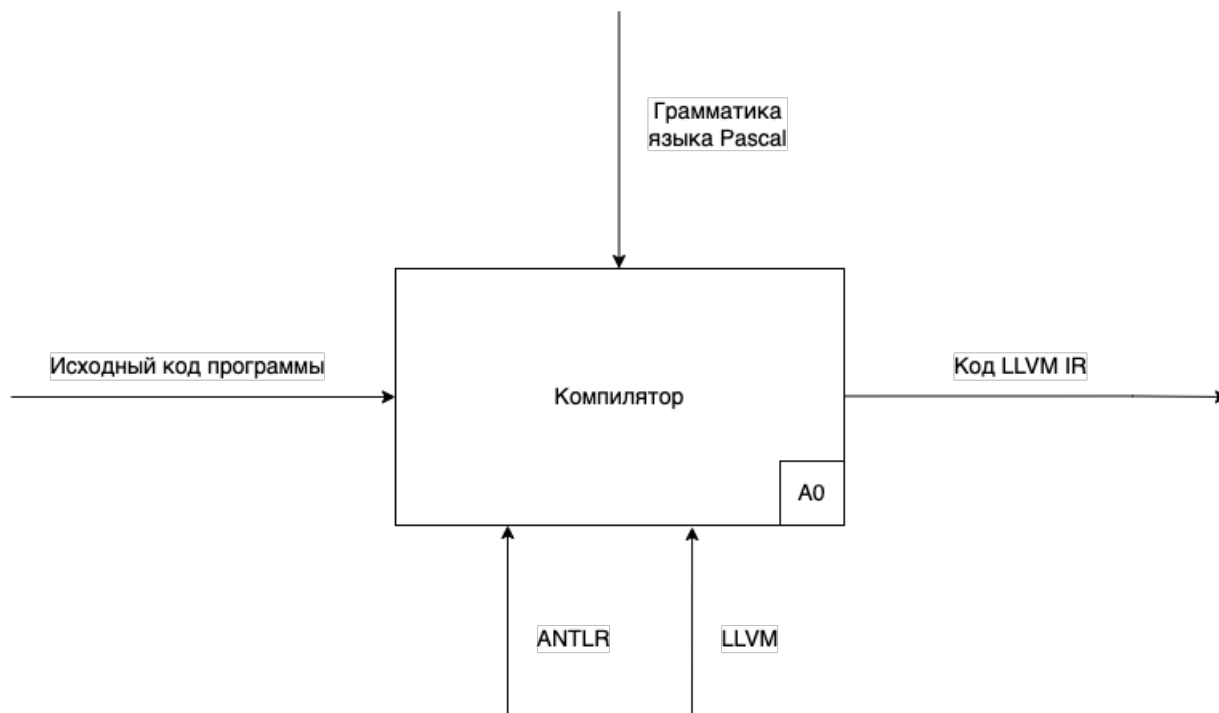


Рисунок 2.1: Концептуальная модель в нотации IDEF0

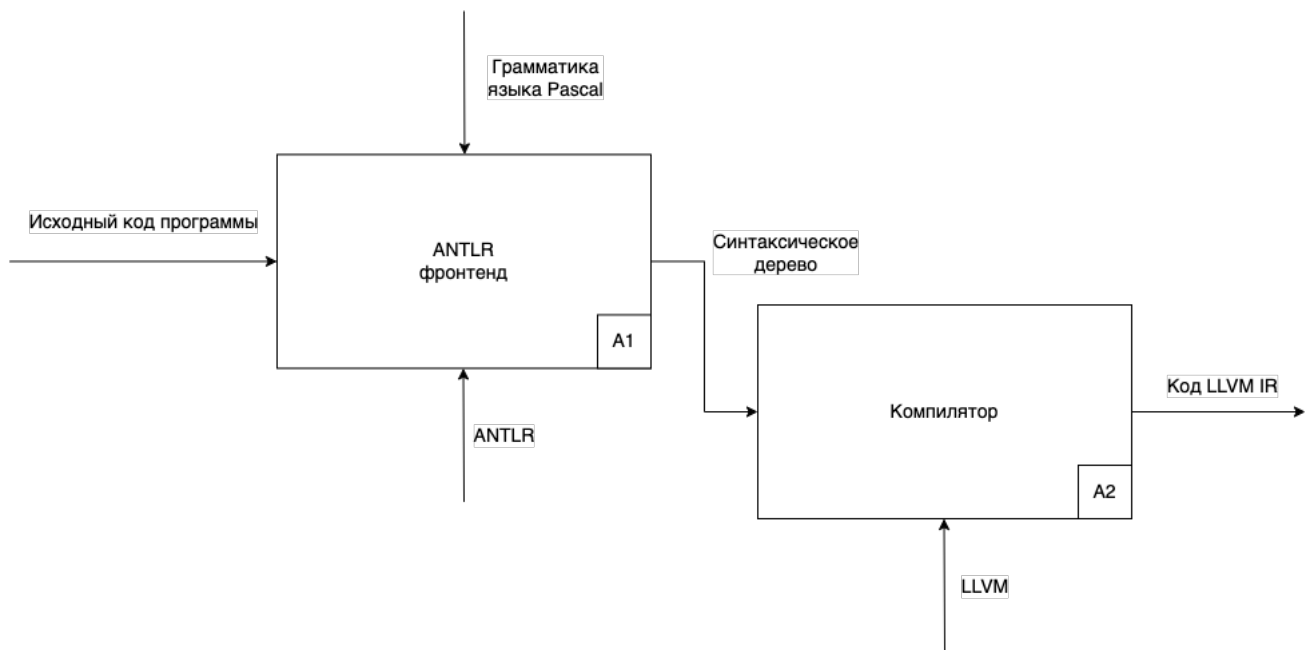


Рисунок 2.2: Детализированная концептуальная модель в нотации IDEF0

2.1.1 Поддерживаемые типы данных

Была предложена поддержка следующих базовых типов.

- *IntegerType* – целое число длины `size` бит, обладающее знаком согласно флагу `isSigned`;
- *DoubleType* – число с плавающей запятой длины `size` бит;
- *StructType* – структура, содержащая поля с именами `FieldNames` и типами `FieldsTypes`;
- *ArrayType* – массив, содержащий элементы типа `type`;
- *PointerType* – указатель на тип `type`;
- *FunctionType* – функция, имеющая аргументы типы `args` и возвращаемый тип `retType`.

Данного представления достаточно, чтобы поддерживать базовые типы языка: *boolean*, *char*, *real*, *integer*, *string* – создавать пользовательские структуры (*record*), процедуры и функции.

2.2 Построение анализаторов исходного кода

В данной работе лексический и синтаксический анализаторы создаются при помощи ANTLR. На вход принимается грамматика языка программирования в формате ANTLR4. Она состоит из объявления ключевых слов и правил вывода.

В результате работы ANTLR4 генерируются файлы, содержащие классы лексического и синтаксического анализаторов, а также необходимые вспомогательные файлы и классы для их функционирования. Кроме того, создаются шаблоны классов для обхода дерева разбора, полученного в результате работы парсера.

Лексер принимает на вход текст программы, преобразованный в поток символов, и выдает поток токенов, который передается на вход парсера. После выполнения парсера создается дерево разбора. Любые ошибки, возникающие в процессе работы лексера и парсера, выводятся в стандартный поток ввода-вывода.

Абстрактное синтаксическое дерево (AST) предоставляет структурированное представление программы. Его можно обойти двумя основными способами: с помощью паттерна Listener и паттерна Visitor.

Паттерн Listener позволяет обходить дерево в глубину, вызывая обработчики соответствующих событий при входе и выходе из узла дерева. Когда происходит обход дерева, при входе в узел вызывается соответствующий обработчик, а при выходе - другой обработчик. Это позволяет реализовать обработку узлов на основе их типов и состояний.

Паттерн Visitor предоставляет более гибкий способ обхода дерева. Он позволяет определить, какие узлы и в каком порядке нужно посетить. Для каждого типа узла в дереве реализуется метод его посещения. Обход начинается с точки входа в программу, обычно с корневого узла. Каждый узел посещается в заданном порядке, что позволяет реализовать различные алгоритмы обработки AST-дерева.

2.3 Построение промежуточного представления (IR)

В данной работе был использован паттерн Visitor для обхода абстрактного синтаксического дерева. Данный подход обеспечивает гибкость и позволяет точно контролировать порядок посещения узлов дерева, что удобно при реализации различных анализов и преобразований.

В структуре проекта есть два основных класса: IRBuilder, который отвечает за создание LLVM IR кода, и BaseContext, который является адаптером для IRBuilder и предоставляет интерфейс для создания промежуточного представления кода Pascal. От BaseContext наследуются классы ModuleContext и ProcedureContext: первый предоставляет интерфейс для описания контекста модуля (глобального контекста), а второй служит для описания контекста внутри процедуры или функции. Класс ParserVisitor представляет собой посетителя, который обходит абстрактное синтаксическое дерево (класс AST) и выполняет различные операции над его узлами. Он расширяет базовый посетитель PascalVisitor, чтобы обеспечить специфическую функциональность для языка Pascal. Связь между ParserVisitor и ModuleContext, ProcedureContext показывает, что посетитель использует их для создания кода внутри методов обхода AST-дерева.

На рисунках 2.3-2.4 представлена схема алгоритма построения промежуточного представления для цикла repeat-until.

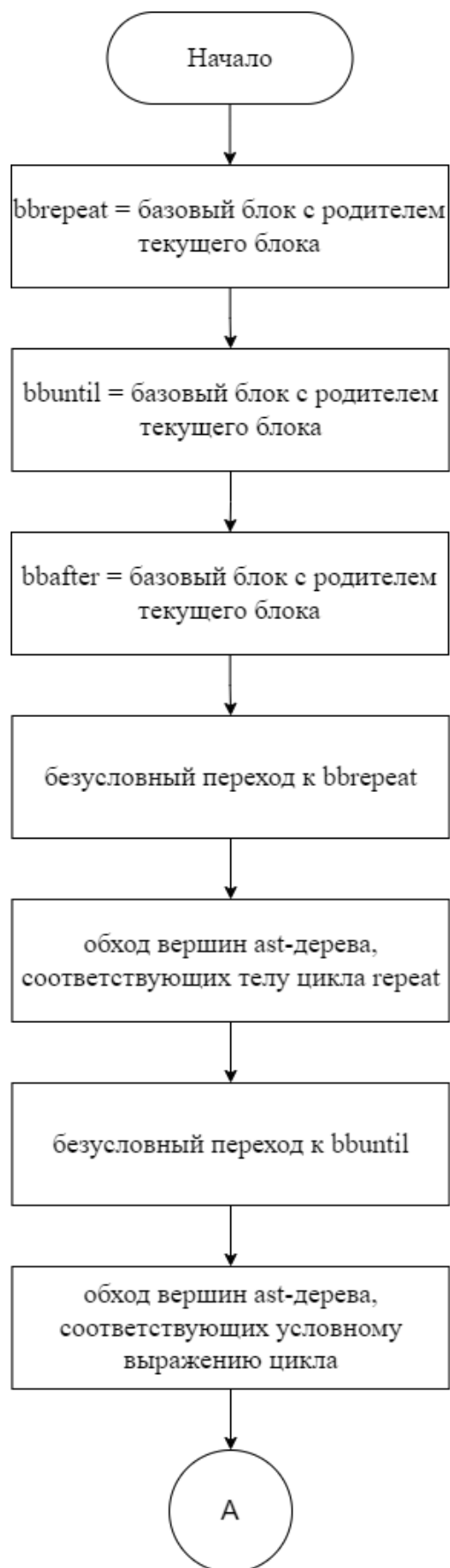


Рисунок 2.3: Схема алгоритма построения IR для цикла repeat-until



Рисунок 2.4: Схема алгоритма построения IR для цикла repeat-until (окончание)

3 Технологическая часть

3.1 Выбор средств программной реализации

В ходе разработки было решено использовать язык программирования *Python*, так как он предоставляет широкие возможности для реализации приложений различного типа.

Язык программирования *Python* через стороннюю библиотеку *LLVMLite* обладает возможностью эмуляции LLVM кода без использования сторонних утилит, что обеспечивает высокую производительность и эффективность разработки.

Кроме того, ANTLR4, используемый для генерации кода лексера и парсера, также способен порождать код на языке *Python*, что позволяет легко интегрировать сгенерированный код.

Для обеспечения удобного импорта и экспорта данных о структуре пакетов была выбрана библиотека *Boost*, обладающая нужными инструментами для работы с данными.

Для написания кода были использованы интегрированная среда разработки Pycharm.

3.2 Сгенерированные классы анализаторов

В результате работы ANTLR генерируются следующие файлы.

- Pascal.interp содержит данные (таблицы предсказания, множества следования, информация о правилах грамматики и т.д.) для интерпретатора ANTLR, используются для ускорения работы сгенерированного парсера для принятия решений о разборе входного потока.
- Pascal.tokens и PascalLexer.tokens перечислены символические имена токенов, каждому из которых сопоставлено числовое значение типа токена. ANTLR4 использует их для создания отображения между символическими именами токенов и их числовыми значениями.
- Интерфейсы PascalListener.py и PascalVisitor.py.

- PascalBaseListener.py и PascalBaseVisitor.py – реализации соответствующих интерфейсов паттернов Listener и Visitor.
- PascalParser.py – синтаксический анализатор.
- PascalLexer.py – лексический анализатор.

3.3 Тестирование компилятора

Были созданы программы, которые соответствуют измененной грамматике языка. Они предназначены для проверки правильности работы различных конструкций языка. Данные программы охватывают следующие сценарии:

- создание переменных и констант;
- ввод и вывод;
- работа с целыми и вещественными числами;
- создание функций и работа с рекурсией;
- создание ветвлений программы;
- создание циклов, выход из цикла;
- работа с массивами фиксированного размера;
- работа со структурами;
- работа с указателями;
- логические операции, ленивые логические вычисления.

3.4 Пример программы инверсии списка

В листингах 3.1-3.2 представлен пример программы инверсии списка.

```

program fibo;

var n, m: integer;

function fibo_calc(n: integer): integer;
begin
    if (n = 1) or (n = 2) then begin
        fibo_calc := 1;
    end else begin
        fibo_calc := fibo_calc(n - 1) + fibo_calc(n - 2);
    end;
end;

begin
    randomize;
    n := random mod 35;
    writeln('fibo(', n, ') = ', fibo_calc(n));
end.

```

Листинг 3.1: Пример программы инверсии списка на языке Pascal

```

; ModuleID = "fibo"
target triple = "unknown-unknown-unknown"
target datalayout = ""

declare i32 @"printf"(i8* %".1", ...)

declare i32 @"scanf"(i8* %".1", ...)

declare i64 @"rand"()

declare i64 @"time"(i1* %".1")

declare void @"srand"(i64 %".1")

declare i8* @"realloc"(i8* %".1", i64 %".2")

@"n" = private global i64 0
@"m" = private global i64 0
define i64 @"fibo_calc"(i64 %".1")
{
entry:
    %"n" = alloca i64
    store i64 %".1", i64* %"n"
    %"fibo_calc_ret" = alloca i64
    store i64 0, i64* %"fibo_calc_ret"
    %".5" = getelementptr i64, i64* %"n", i32 0
    %".6" = load i64, i64* %".5"
    %".7" = icmp eq i64 %".6", 1
    %".8" = getelementptr i64, i64* %"n", i32 0
    %".9" = load i64, i64* %".8"
    %".10" = icmp eq i64 %".9", 2

```

```

    %".11" = or i1 %".7", %".10"
    br i1 %".11", label %"entry.if", label %"entry.else"
entry.if:
    %".13" = getelementptr i64, i64* %"fibo_calc_ret", i32 0
    %".14" = load i64, i64* %".13"
    store i64 1, i64* %".13"
    br label %"entry.endif"
entry.else:
    %".17" = getelementptr i64, i64* %"fibo_calc_ret", i32 0
    %".18" = load i64, i64* %".17"
    %".19" = getelementptr i64, i64* %"n", i32 0
    %".20" = load i64, i64* %".19"
    %".21" = sub i64 %".20", 1
    %".22" = call i64 @"fibo_calc"(i64 %".21")
    %".23" = getelementptr i64, i64* %"n", i32 0
    %".24" = load i64, i64* %".23"
    %".25" = sub i64 %".24", 2
    %".26" = call i64 @"fibo_calc"(i64 %".25")
    %".27" = add i64 %".22", %".26"
    store i64 %".27", i64* %".17"
    br label %"entry.endif"
entry.endif:
    %".30" = getelementptr i64, i64* %"fibo_calc_ret", i32 0
    %".31" = load i64, i64* %".30"
    ret i64 %".31"
}

define i32 @"fibo_main"()
{
entry:
    %".2" = call i64 @"time"(i1* null)
    call void @"srand"(i64 %".2")
    %".4" = getelementptr i64, i64* @"n", i32 0
    %".5" = load i64, i64* %".4"
    %".6" = call i64 @"rand"()
    %".7" = srem i64 %".6", 35
    store i64 %".7", i64* %".4"
    store i8* getelementptr ([6 x i8], [6 x i8]* @".literal_1", i32 0, i32 0),
        i8** @".literal_ptr_2"
    %".10" = load i8*, i8** @".literal_ptr_2"
    %".11" = getelementptr i64, i64* @"n", i32 0
    %".12" = load i64, i64* %".11"
    store i8* getelementptr ([5 x i8], [5 x i8]* @".literal_3", i32 0, i32 0),
        i8** @".literal_ptr_4"
    %".14" = load i8*, i8** @".literal_ptr_4"
    %".15" = getelementptr i64, i64* @"n", i32 0
    %".16" = load i64, i64* %".15"
    %".17" = call i64 @"fibo_calc"(i64 %".16")

```

```

%.18" = alloca [10 x i8]
store [10 x i8] c"%s%d%s%d\0a\00", [10 x i8]* %.18"
%.20" = alloca i8*
%.21" = getelementptr [10 x i8], [10 x i8]* %.18", i32 0, i32 0
store i8* %.21", i8** %.20"
%.23" = load i8*, i8** %.20"
%.24" = call i32 @printf(i8* %.23", i8* %.10", i64 %.12",
    i8* %.14", i64 %.17")
ret i32 0
}

@".literal_1" = global [6 x i8] c"fibonacci\00"
@".literal_ptr_2" = global i8* null
@".literal_3" = global [5 x i8] c") = \00"
@".literal_ptr_4" = global i8* null

```

Листинг 3.2: IR-представление программы инверсии списка для LLVM

Компилятор справился со всеми задачами, описанными выше, успешно.

ЗАКЛЮЧЕНИЕ

В процессе выполнения курсовой работы были рассмотрены основные части компилятора, принцип их работы, алгоритмы и способы их реализации. Также были рассмотрены инструменты генерации лексических и синтаксических анализаторов.

В результате проведенной работы был осуществлен анализ предметной области разработки компилятора языка программирования Pascal. Были рассмотрены ключевые аспекты, включая анализ платформ, сравнительное изучение инструментов для генерации лексических и синтаксических анализаторов, а также аспекты типизации в данном языке.

Цель курсовой работы была достигнута. Все поставленные задачи были выполнены в полном объеме: был разработан прототип компилятора Pascal.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Компиляторы: принципы, технологии и инструментарий / Ахо А, Лам М, Сети Рави и Ульман Дж // М.: Вильямс. – 2008. – Т. 1. – С. 257.
2. О языке Pascal. [Электронный ресурс] . Режим доступа: <https://pas1.ru> (Дата обращения: 15.08.2024).
3. Reinhard W., Dieter M. Compiler design // Addison-Wesley Publishing Company – 1995. – С. 132-135.
4. Ахо А., Ульман Дж. Теория синтаксического анализа, перевода и компиляции. // Москва : Мир – 1987. – С. 56-71.
5. Кадомский А. А., Сабинин О. Ю. Исследование возможности создания универсального языка программирования высокого уровня // Theoretical & Applied Science. – 2018. – № 5. – С. 84–90.
6. Lesk Michael E, Schmidt Eric. Lex: A lexical analyzer generator // Bell Laboratories Murray Hill, NJ – 1975. – № 39
7. Paxson Vern. Flex – A fast scanner generator Edition [Электронный ресурс] — 1995. — Режим доступа: https://ftp.gnu.org/old-gnu/Manuals/flex-2.5.4/html_mono/flex.html (дата обращения: 20.04.2024).
8. Parr Terence. The definitive ANTLR 4 reference // The Definitive ANTLR 4 Reference. – 2013. – С. 1–326.
9. The Compiler Generator Coco/R [Электронный ресурс]. – Режим доступа: <https://ssw.jku.at/Research/Projects/Coco/> (Дата обращения: 20.04.2024).
10. Foundation LLVM. The LLVM Compiler Infrastructure Project [Электронный ресурс]. – 2024. – Режим доступа: <https://llvm.org/> (Дата обращения: 21.04.2024).

ПРИЛОЖЕНИЕ А.

(Грамматика языка Pascal)

```
AND: 'AND' ;
ARRAY: 'ARRAY' ;
BEGIN: 'BEGIN' ;
BOOLEAN: 'BOOLEAN' ;
CASE: 'CASE' ;
CHAR: 'CHAR' ;
CHR: 'CHR' ;
CONST: 'CONST' ;
DIV: 'DIV' ;
DO: 'DO' ;
DOWNTO: 'DOWNTO' ;
ELSE: 'ELSE' ;
END: 'END' ;
FILE: 'FILE' ;
FOR: 'FOR' ;
FUNCTION: 'FUNCTION' ;
GOTO: 'GOTO' ;
IF: 'IF' ;
IN: 'IN' ;
INTEGER: 'INTEGER' ;
LABEL: 'LABEL' ;
MOD: 'MOD' ;
NIL: 'NIL' ;
NOT: 'NOT' ;
OF: 'OF' ;
OR: 'OR' ;
PACKED: 'PACKED' ;
PROCEDURE: 'PROCEDURE' ;
PROGRAM: 'PROGRAM' ;
REAL: 'REAL' ;
RECORD: 'RECORD' ;
REPEAT: 'REPEAT' ;
SET: 'SET' ;
THEN: 'THEN' ;
TO: 'TO' ;
TYPE: 'TYPE' ;
UNTIL: 'UNTIL' ;
VAR: 'VAR' ;
WHILE: 'WHILE' ;
WITH: 'WITH' ;
PLUS: '+' ;
MINUS: '-' ;
STAR: '*' ;
SLASH: '/' ;
ASSIGN: ':=' ;
COMMA: ',' ;
SEMI: ';' ;
COLON: ':' ;
EQUAL: '=' ;
NOT_EQUAL: '<>' ;
LT: '<' ;
LE: '<=' ;
GE: '>=' ;
GT: '>' ;
LPAREN: '(' ;
```

```

RPAREN: ')' ;
LBRACK: '[' ;
LBRACK2: '(' ;
RBRACK: ']' ;
RBRACK2: ')' ;
POINTER: '^' ;
AT: '@' ;
DOT: '.' ;
DOTDOT: '..' ;
LCURLY: '{' ;
RCURLY: '}' ;
UNIT: 'UNIT' ;
INTERFACE: 'INTERFACE' ;
USES: 'USES' ;
STRING: 'STRING' ;
IMPLEMENTATION: 'IMPLEMENTATION' ;
TRUE: 'TRUE' ;
FALSE: 'FALSE' ;
WS: [ \t\r\n ] -> skip ;
COMMENT_1: '(*.*?*)' -> skip ;
COMMENT_2: '{.*?}' -> skip ;
IDENT: ('A' .. 'Z') ('A' .. 'Z' | '0' .. '9' | '_' ) * ;
STRING_LITERAL: '\' ('\' | ~ ('\'')) * '\' ;
NUM_INT: ('0' .. '9') + ;
NUM_REAL: ('0' .. '9') + (( '.' ('0' .. '9') + (EXPONENT) ) ? | EXPONENT ) ;
fragment EXPONENT: ('E') ('+' | '-' ) ? ('0' .. '9') + ;

```

Листинг 3.3: Описание лексического анализатора

```

grammar pascal;
options { caseInsensitive = true; }
program: programHeading block DOT EOF ;
programHeading: PROGRAM identifier SEMI ;
identifier: IDENT ;
block: (labelDeclarationPart | constantDefinitionPart | typeDefinitionPart |
      variableDeclarationPart | procedureAndFunctionDeclarationPart)* compoundStatement ;
labelDeclarationPart: LABEL label (COMMA label)* SEMI ;
label: unsignedInteger ;
constantDefinitionPart: CONST (constantDefinition SEMI)+ ;
constantDefinition: identifier EQUAL constant ;
constantChr: CHR LPAREN unsignedInteger RPAREN ;
constant: unsignedNumber | sign unsignedNumber | identifier | sign identifier | string |
      constantChr ;
unsignedNumber: unsignedInteger | unsignedReal ;
unsignedInteger: NUM_INT ;
unsignedReal : NUM_REAL ;
sign: PLUS | MINUS ;
bool_: TRUE | FALSE ;
string: STRING_LITERAL ;
typeDefinitionPart: TYPE (typeDefinition SEMI)+ ;
typeDefinition: identifier EQUAL (type_ | functionType | procedureType) ;
functionType: FUNCTION (formalParameterList)? COLON resultType ;
procedureType: PROCEDURE (formalParameterList)? ;
type_: simpleType | structuredType | pointerType ;
simpleType: scalarType | subrangeType | typeIdentifier | stringtype ;
scalarType: LPAREN identifierList RPAREN ;
subrangeType: constant DOTDOT constant ;
typeIdentifier: identifier | (CHAR | BOOLEAN | INTEGER | REAL | STRING) ;
structuredType: unpackedStructuredType ;
unpackedStructuredType: arrayType | recordType ;
stringtype: STRING LBRACK (identifier | unsignedNumber) RBRACK ;

```

```

arrayType: ARRAY LBRACK typeList RBRACK OF componentType | ARRAY LBRACK2 typeList RBRACK2 OF
    componentType ;
typeList: indexType (COMMA indexType)* ;
indexType: simpleType ;
componentType: type_ ;
recordType: RECORD fieldList? END ;
fieldList: fixedPart ;
fixedPart: recordSection (SEMI recordSection)* ;
recordSection: identifierList COLON type_ ;
pointerType: POINTER typeIdentifier ;
variableDeclarationPart: VAR variableDeclaration (SEMI variableDeclaration)* SEMI ;
variableDeclaration: identifierList COLON type_ ;
procedureAndFunctionDeclarationPart: procedureOrFunctionDeclaration SEMI ;
procedureOrFunctionDeclaration: procedureDeclaration | functionDeclaration ;
procedureDeclaration: PROCEDURE identifier (formalParameterList)? SEMI block ;
formalParameterList: LPAREN formalParameterSection (SEMI formalParameterSection)* RPAREN ;
formalParameterSection: parameterGroup ;
parameterGroup: identifierList COLON typeIdentifier ;
identifierList: identifier (COMMA identifier)* ;
constList: constant (COMMA constant)* ;
functionDeclaration : FUNCTION identifier (formalParameterList)? COLON resultType SEMI block ;
resultType: typeIdentifier ;
statement: label COLON unlabelledStatement | unlabelledStatement ;
unlabelledStatement: simpleStatement | structuredStatement ;
simpleStatement: assignmentStatement | procedureStatement | gotoStatement | emptyStatement_ ;
assignmentStatement : variable ASSIGN expression ;
variable: (AT identifier | identifier) (LBRACK expression (COMMA expression)* RBRACK | LBRACK2
    expression (COMMA expression)* RBRACK2 | DOT identifier | POINTER)* ;
expression: simpleExpression (relationaloperator expression)? ;
relationaloperator: EQUAL | NOT_EQUAL | LT | LE | GE | GT | IN ;
simpleExpression: term (additiveoperator simpleExpression)? ;
additiveoperator: PLUS | MINUS | OR ;
term: signedFactor (multiplicativeoperator term)? ;
multiplicativeoperator: STAR | SLASH | DIV | MOD | AND ;
signedFactor: (PLUS | MINUS)? factor ;
factor: variable | LPAREN expression RPAREN | functionDesignator | unsignedConstant | set_ |
    NOT factor | bool_ ;
unsignedConstant: unsignedNumber | constantChr | string | NIL ;
functionDesignator: identifier LPAREN parameterList RPAREN ;
parameterList: actualParameter (COMMA actualParameter)* ;
set_: LBRACK elementList RBRACK | LBRACK2 elementList RBRACK2 ;
elementList: element (COMMA element)* | ;
element: expression (DOTDOT expression)? ;
procedureStatement: identifier (LPAREN parameterList RPAREN)? ;
actualParameter: expression parameterwidth* ;
parameterwidth: COLON expression ;
gotoStatement: GOTO label ;
emptyStatement_: ;
empty_: /* empty */ ;
structuredStatement: compoundStatement | conditionalStatement | repetitiveStatement |
    withStatement ;
compoundStatement: BEGIN statements END ;
statements: statement (SEMI statement)* ;
conditionalStatement: ifStatement | caseStatement ;
ifStatement: IF expression THEN statement (: ELSE statement)? ;
caseStatement : CASE expression OF caseListElement (SEMI caseListElement)* (SEMI ELSE
    statements)? END ;
caseListElement : constList COLON statement ;
repetitiveStatement: whileStatement | repeatStatement | forStatement ;
whileStatement: WHILE expression DO statement ;
repeatStatement: REPEAT statements UNTIL expression ;

```

```
forStatement: FOR identifier ASSIGN forList DO statement ;  
forList: initialValue (TO | DOWNTO) finalValue ;  
initialValue: expression ;  
finalValue: expression ;  
withStatement: WITH recordVariableList DO statement ;  
recordVariableList: variable (COMMA variable)* ;
```

Листинг 3.4: Описание синтаксического анализатора